

Generate a Jupyter Notebook that uses PyTorch to implement a custom CNN for the CIFAR-100 dataset without any data augmentation. The model must handle multi-task classification for both the 100 'fine' classes and the 20 'coarse' superclasses. Train the model and provide a performance evaluation.

Check for relevant skills >

Now let me build the Jupyter Notebook with all the required components:

Created a file, read a file >

The notebook is ready. Here's a breakdown of what's inside:

Architecture: MultiTaskCNN

A shared 3-block CNN backbone branches into two independent heads:

Component	Detail
Block 1	Conv→BN→ReLU×2 → MaxPool → 64×16×16
Block 2	Conv→BN→ReLU×2 → MaxPool → 128×8×8
Block 3	Conv→BN→ReLU×2 → MaxPool → 256×4×4
Fine Head	FC(4096→512) → ReLU → Dropout → FC(512→100)
Coarse Head	FC(4096→256) → ReLU → Dropout → FC(256→20)

Training Setup



Reply...

 Multi-task CNN for CIFAR-100 classification 

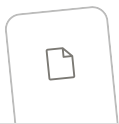
Share

Training Setup

- **Loss:** $L = 1.0 \times \text{CrossEntropy}(\text{fine}) + 0.5 \times \text{CrossEntropy}(\text{coarse})$
- **Optimiser:** AdamW with weight decay
- **Scheduler:** Cosine annealing over 40 epochs
- **Regularisation:** Batch Norm + Dropout (p=0.4) + gradient clipping

Evaluation Sections

1. **Training curves** — loss, fine/coarse accuracy, LR schedule
2. **Top-1 & Top-5** accuracy for fine classes; Top-1 for coarse
3. **Per-superclass bar chart** — which of the 20 superclasses are hardest
4. **Coarse confusion matrix** — full 20×20 row-normalised heatmap
5. **Fine confusion matrix** — 20 most-confused fine classes
6. **Per-class accuracy histogram** — distribution across all 100 fine classes
7. **Qualitative predictions** — colour-bordered grid (green = both correct, yellow = coarse only, red = both wrong)
8. **Full classification report** — precision, recall, F1 per coarse class



Cifar100 multitask cnn
IPYNB

Download



Reply...

Sonnet 4.6 

Claude is AI and can make mistakes. Please double-check responses.